WinDepends – File Analysis Flow

Scope: GUI application (`src/WinDepends`) as of November 2026. Covers the GUI open flow (executable and session file), the shared analysis service, the render step, cancellation, and the CLI flow.

---

1. Overview

Both entry points (GUI and CLI) run dependency analysis through one routine: `CDependsAnalysisService.AnalyzeModel`. It analyzes modules in place on the `CDepends` model and builds no UI. UI trees are built separately from the analyzed model.

Stage	Method	Thread	UI access
Orchestration	`CFileOpenOrchestrationService.ExecuteAsync`	UI	yes
Analysis	`CDependsAnalysisService.AnalyzeModel`	worker (GUI) / main (CLI)	no
Render (GUI)	`CDependsAnalysisService.PopulateModuleTree`	UI	yes
Export (CLI)	`CExporter.Export`	main	no

```
GUI:  OpenInputFileAsync -> ExecuteAsync -> OpenInputFileInternalAsync
        -> Task.Run(AnalyzeModel)  [worker]
        -> RenderAnalyzedModel -> PopulateModuleTree  [UI]
CLI:  Program.Main -> CCliHandler.Run -> RunAnalysis
        -> AnalyzeModel  [main thread]
        -> CExporter.Export
```

---

2. Components

Component	File	Role
`Program`	`Program.cs`	Chooses CLI (`CCliHandler.ShouldRunAsCli`) or GUI (`Application.Run(new MainForm())`).
`CFileOpenOrchestrationService`	`MainForm/CFileOpenState.cs`	UI lock, .lnk resolution, result and message mapping, error handling.

Component	File	Role
`MainForm` (partials)	`MainForm/*.cs`	Open flow, render, cancellation, progress dispatch.
`CDependsAnalysisService`	`Analysis/CDependsAnalysisService.cs`	`AnalyzeModel`, `PopulateModuleTree`, traversal, per-module processing.
`CDependsAnalysisContext`	`Analysis/CDependsAnalysisContracts.cs`	Configuration snapshot, root, imports table, log callback, progress callback, `CancellationToken`.
`CDependsModelAnalysisRequest` / `Result`	same	Input (`CDepends`, file open settings, context) and counts.
`CDependsTreePopulationRequest` / `Result`	same	Render request and result (root node, counts).
`CDependsAnalysisScope`	same	Owns the `CActCtxHelper` for one analysis.
`CDependsDuplicateModuleState`	`Analysis/CDependsDuplicateModuleState.cs`	Copies state from the canonical module to a duplicate; error-propagation rule.
`CCoreClient`	`CCoreClient/*.cs`	TCP client to the core server: open, headers, imports, exports, forwarders, stats.
`CPathResolver`	`Resolution/`	Module path resolution (process-static state).
`CAssemblyRefAnalyzer`	`Resolution/CAssemblyRefAnalyzer.cs`	.NET assembly reference resolution (experimental).
`CCliHandler`, `CExporter`	`Cli/`	CLI parsing, run, export.

---

3. Key concepts

3.1 Analysis context

`CDependsAnalysisContext` carries everything a worker needs, so it does not touch the form:

- `Configuration`: a **snapshot copy** of `CConfiguration` (GUI) or the loaded and adjusted config (CLI).
- `RootModule`, `ParentImportsHashTable`.

- `AddLogMessage`: `AppLogger.LogExt` (GUI) or a console callback (CLI).
- `ReportProgress`: optional.
- `CancellationToken`.

3.2 Traversal order (decides the canonical instance)

`TraverseModelCore` (analysis) and `PopulateTreeCore` (render) use the same order:

1. Root (depth 0).
2. All direct dependents of the root (depth 1).
3. Then, for each depth-1 module in order, a depth-first walk of its dependents (depth 2 and deeper).

The first module seen with a given file name (case-insensitive) is the **canonical instance**. Later ones are **duplicates**.

3.3 Duplicates

For a duplicate, `CDependsDuplicateModuleState.Apply` copies file state from the canonical module (`FileNotFound`, `IsInvalid`, `ExportContainErrors`, `IsDotNetModule`, `ModuleData` copy) and sets `OriginalInstanceId` to the canonical `InstanceId`. A duplicate is **not opened** and gets no dependents. `OtherErrorsPresent` is deliberately not copied; instead, if the canonical module has genuine errors (and is not an API-set contract or stopped node), the **parent** is flagged (`ShouldPropagateErrors`).

`OriginalInstanceId == 0` means "this module is canonical".

3.4 InstanceId

Set in `CDependsAnalysisService.ProcessModule` as `module.GetHashCode()` (`OrdinalIgnoreCase` hash of `FileName`). The UI looks modules up by it. Collisions are theoretically possible (about 1 in 2^32 per pair) and are a known pre-existing risk.

3.5 Depth rule

A non-root module is analyzed only if its parent's `Depth <= ModuleNodeDepthMax`. Modules beyond that limit are counted as processed but are not opened. The render step applies the same limit through `ValidateTreeDepth`.

3.6 Concurrency model

- Only one analysis may run: `_openInputFileInProgress` in the form, plus the static `s_analysisInProgress` guard in the service (`EnterAnalysis`).
- `CPathResolver` has process-static mutable state (`ActCtxHelper`, `Initialized`, known DLLs, user directories).
- `CCoreClient` has one shared TCP transport and no locking. Never dispose or use it from another thread while a worker is active.

---

4. GUI flow: opening an executable

4.1 Entry

`OpenFileHandler`, drag and drop, MRU and `ViewRefreshItem_Click` call `OpenInputFileAsync(fileName)`:

1. If `_openInputFileInProgress` is set, log a warning and return `false`.
2. Set `_openInputFileInProgress = true` and create `_openCts`.
3. `await CFileOpenOrchestrationService.ExecuteAsync(state, OpenInputFileInternalAsync, UpdateOperationStatus, SetOpenInputUiEnabled, focusTreeView)`.
4. `finally`: dispose `_openCts`, clear flags, and `BeginInvoke(Close)` if a close was requested during the operation.

4.2 Orchestration (`ExecuteAsync`)

1. `setUiEnabled(false)`: disables menu, toolbar, tree view and list view.
2. `ResolveShortcut` follows a `.lnk` to its target.
3. `state.Result = await openInputFile(resolvedFileName)`.
4. `IsSuccess` is true for `Success` and `SuccessSession`. `PopulateResultMessage` builds the log text. The result is logged and shown in the status bar.
5. Exceptions:
   - `OperationCanceledException`: result `Cancelled`, normal "cancelled" message, no error entry (safety net for cancellations that escape the open flow).
   - Any other `Exception`: result `Failure`, generic error message, full exception text logged.
6. `finally`: `setUiEnabled(true)` and focus the tree view.

4.3 `OpenInputFileInternalAsync` (non-session branch)

1. If `UseAsDefault` is off, show `FileOpenForm`. If the user cancels the dialog, return `Cancelled`. If `UseAsDefault` is checked, write the chosen settings back into `_configuration`.
2. `CloseInputFile()`, log "Opening ... for analysis", optionally clear the log.
3. Take a **snapshot**: `analysisConfiguration = new(_configuration)`; create an empty imports hash table.
4. Create `_depends = new CDepends(fileName)` and store `SessionNodeMaxDepth`.
5. Build `CDependsAnalysisContext` with the snapshot, log callback (`AppLogger.LogExt`), `ReportAnalysisProgress` and the token from `_openCts`.
6. Build `CDependsModelAnalysisRequest`.

7. `await Task.Run(() => _analysisService.AnalyzeModel(request), token)`.
   - `OperationCanceledException`: `CloseInputFile()` and return `Cancelled`.
8. Copy the analysis imports table into `_parentImportsHashTable` (UI thread).
9. Render (section 6), then update the list view and sort.
10. On success: `PostOpenFileUpdateControls`, optional `ExpandAllModulesWithUpdate`, enable Save and Copy, select the first node.
11. Return `Success`, `SuccessSession` (session files) or `Failure`.

---

5. Analysis phase (`AnalyzeModel`)

5.1 Steps

```
AnalyzeModel(request)
  EnterAnalysis()                        // static single-run guard
  BeginAnalysis(rootFileName, context)
      CPathResolver.Initialized = false
      CAssemblyRefAnalyzer.ClearCache()  // once per analysis
      CActCtxHelper = new(rootFileName); CPathResolver.ActCtxHelper = it
      returns CDependsAnalysisScope      // disposes the activation context
  TraverseModelCore(root, settings, context, metrics.ProcessModule, progress)
  return CDependsModelAnalysisResult(depends, processedCount, duplicateCount)
```

The scope and the guard are released even on exceptions (`using`).

5.2 Traversal (`TraverseModelCore`)

For each visited module it calls `metrics.ProcessModule` (section 5.3), then recurses into the dependents if the visit result allows. Progress `Populating` is reported per module (root, depth 1, then deeper levels).

5.3 `CDependsModelAnalysisMetrics.ProcessModule`

1. `context.CancellationToken.ThrowIfCancellationRequested()`: the **cancellation check point**, once per module.
2. Count the module as processed.
3. Depth rule (section 3.5). If beyond the limit, return `Stop`.
4. Look up the module in `_canonicalModules` (`Dictionary<string, CModule>`, `OrdinalIgnoreCase`, O(1)).
5. **Duplicate:** `CDependsDuplicateModuleState.Apply`, propagate errors to the parent if required, increment the duplicate counter.
6. **Canonical:** `CDependsAnalysisService.ProcessModule` (section 5.4), then add the module to `_canonicalModules`.
7. If the module is a forwarded module whose file was not found, flag the parent `OtherErrorsPresent`.

8. Set `module.Depth = depth` and return `Continue`.

5.4 `ProcessModule` (opens one canonical module)

1. Copy the file open settings. For a dependency, if `PropagateSettingsOnDependencies` is off, reset `ProcessRelocsForImage`, `UseStats` and `UseCustomImageBase`.
2. `module.InstanceId = module.GetHashCode()`.
3. Progress `OpeningModule`.
4. `_coreClient.OpenModule(ref module, settings)`.
5. On failure, progress `ModuleProcessingFailed`.
6. `HandleModuleOpenStatus(status)`:
   - `Okay`: `HandleOpenedModule`.
   - Each error status logs a specific message. `ErrorFileNotFound` and `ErrorInvalidHeadersOrSignatures` have dedicated handlers (delay-load and API-set extension variants).

5.5 `HandleOpenedModule`

In order:

1. Progress `ReadingHeaders`; `GetModuleHeadersInformation`; sets `IsProcessed`.
2. Root only: `CPathResolver.QueryFileInformation(module)`.
3. Progress `ReadingImportsAndExports`; `GetModuleImportExportInformation` (section 5.6) with the configuration's user-mode and kernel-mode search order.
4. If `ExpandForwarders`: progress `ExpandingForwarders`; `ExpandAllForwarderModules` (breadth-first, expands each module's forwarders once, logs forward-target export errors), then `ValidateForwardedExports`.
5. If `UseStats`: progress `ReadingStatistics`; `GetCoreCallStats`.
6. `CloseModule()`; log statistics if collected.
7. Warnings (also set `OtherErrorsPresent` where noted):
   - export errors,
   - unresolved forwarded exports,
   - CPU type differs from the root (sets flag),
   - missing relocations when relocation processing is on (sets flag),
   - module not fully processed.

5.6 `GetModuleImportExportInformation`

1. `ProcessExports`: adds export functions, collects forwarder entries (a local `HashSet` prevents duplicates, self-forwards are skipped), and checks already-collected parent imports against the exports (sets `ExportContainErrors`).

2. Imports: `CheckInvalidImports`, `CheckIfKernelModule`, then `ProcessImports` for normal and delay-load libraries. Each import library goes through `AddDependentModule`: API-set contract resolution, `CPathResolver.ResolvePathForModule` (search order, KnownDlls, activation context), then a new `CModule` is added to `parent.Dependents`. Imported functions are also added to the shared `parentImportsHashTable`.
3. Import exceptions are logged and set `OtherErrorsPresent`.
4. .NET: `IsDotNetModule = ImageDotNet == 1`. If experimental features are enabled, `ProcessNetAssemblies` adds assembly references as dependents via `CAssemblyRefAnalyzer` (caches are cleared once per analysis, not per module).

Because dependents are created while a module is processed, the traversal always discovers the next level from the module it just opened.

---

6. Render phase (GUI)

Runs on the UI thread after the `await` returns.

6.1 Executable

`RenderAnalyzedModel(depends)` builds a `CDependsTreePopulationRequest` and calls `PopulateModuleTree`, which walks the model in the same traversal order (`PopulateTreeCore`). For each module `RenderAnalyzedModule`:

1. `ValidateTreeDepth(parentNode, _configuration.ModuleNodeDepthMax)`; otherwise return `null` (no node).
2. Build the display name (API-set name, full path, upper case options).
3. Choose the icon, create the `TreeNode` (module in `Tag`), and attach it to the parent or the tree root.
4. If `OriginalInstanceId == 0` (canonical): add to `_loadedModulesList` and to the local canonical map. Duplicates are only shown as nodes.

`TVModules.BeginUpdate/EndUpdate` wraps the call. The list view gets `VirtualListSize` and a sort.

6.2 Session file

See section 7. It uses the same `PopulateModuleTree` with a different processor (`ProcessSessionModule`).

---

7. GUI flow: opening a session file (`.wds`)

1. `OpenInputFileInternalAsync` takes the session branch and optionally clears the log.
2. `OpenSessionFileAsync(fileName, token)`:
    1. `CloseInputFile()`, reset `_sessionRenderCounter`.

2. `await Task.Run(LoadSessionObjectFromFile)`: deserialization on a worker thread (`DataContractJsonSerializer`, cannot be interrupted).
3. If the token is cancelled: clear the status text, return `false`.
4. If deserialization failed: return `false`.
5. Render with `PopulateSessionObjectToLists` -> `PopulateModuleTree` with `ProcessSessionModule`:
   - every 256 modules: `Application.DoEvents()`,
   - `_openCts?.Token.ThrowIfCancellationRequested()` per module,
   - `AddSessionModuleEntry` -> `AddModuleEntryCore`.
6. Restore the saved analysis log entries.
7. `catch (OperationCanceledException)`: `CloseInputFile()` (cancel).

3. `AddModuleEntryCore` (session): validate depth, `TryApplyExistingModuleState` (looks the module up in `_loadedModulesByName`; if found it is a duplicate: apply state and propagate errors to the parent node), create the node, and for a new module add it to `_loadedModulesList` and `_loadedModulesByName`.

Session modules are **not re-analyzed**: the saved model is the source of truth. The duplicate decision is recomputed by name during render.

---

8. Progress reporting

- Workers call `ReportProgress`; only the `Populating` stage reaches the UI.
- `ReportAnalysisProgress` is thread-safe: on the UI thread it updates the status bar directly; on a worker it stores the latest progress under `_analysisProgressLock` and posts a **single** `BeginInvoke(DrainAnalysisProgress)` at a time. The newest message wins (coalescing), so the UI queue never floods.
- It skips updates during shutdown or when the handle is gone, and handles a `BeginInvoke` failure during disposal.
- Other stages (`OpeningModule`, `ReadingHeaders`, ...) are used by the CLI console progress and are ignored by the GUI.

---

9. Cancellation

Trigger	Handling
Esc	`ProcessCmdKey`: if `_openCts` exists and is not cancelled, `Cancel()` and show "Cancelling analysis..." (key swallowed).

Trigger	Handling
Closing the form during an open	`FormClosing`: sets `_closeRequested`, cancels, `e.Cancel = true`. `OpenInputFileAsync.finally` then calls `Close()` via `BeginInvoke`.
Ctrl+C (CLI)	`Console.CancelKeyPress` cancels the token and swallows the default termination.

Cooperative cancellation points:

- `ProcessModule` (per module, analysis),
- `ProcessSessionModule` (per module, session render),
- after session deserialization (not interruptible while loading, so Esc takes effect when the load finishes).

An `OperationCanceledException` is converted as follows:

- from `AnalyzeModel`: caught in `OpenInputFileInternalAsync`, which closes the input and returns `Cancelled`;
- from session render: caught in `OpenSessionFileAsync` (`CloseInputFile`);
- anything that escapes: caught in `ExecuteAsync` and reported as `Cancelled` with no error entry.

`CloseInputFile()` clears the tree, lists, `_loadedModulesByName`, and `_depends`. A cancelled analysis may leave `_depends` partially filled until it is closed.

---

10. Result mapping and UI state

`FileOpenResult`: `Success`, `SuccessSession`, `Failure`, `Cancelled`. `PopulateResultMessage` turns each into the log and status text (`Cancelled` and the default case give "has been cancelled").

UI lock: `SetOpenInputUiEnabled(false)` disables menu, toolbar, `TVModules` and `LVModules` for the whole operation. Esc and window close remain available.

---

11. CLI flow

11.1 Detection and parsing

`Program.Main` calls `CCliHandler.ShouldRunAsCli(args)`: any of `-o/--output`, `-f/--format`, `-d/--depth` (including `--depth=`), `-q`, `-e`, `-i`, `-k`, `-h`, `-v`, `-?`, `--no-exports`, `--no-imports`, `--no-resolve`, `--short-paths`. `Run` attaches to the parent console (or allocates one), parses options, and handles `--help` and `--version`. If no output file is given it uses the input path with the format's extension.

11.2 `RunAnalysis`

1. Load the configuration. `maxDepth` = `-d` if given, else `config.ModuleNodeDepthMax`.
2. Apply the options: `ResolveAPIsets`, `FullPaths`, and `config.ModuleNodeDepthMax = maxDepth - 1` (so `-d N` means N dependency levels, matching the old CLI). With `-k` the user-mode search order is replaced by the kernel-mode one.
3. Locate the core server. If missing, print an error and return 1.
4. Create `CCoreClient` with a console log callback (`[*]` / `[!]`, silent with `-q`) and connect (return 1 on failure).
5. Register `Console.CancelKeyPress` to cancel a `CancellationTokenSource`.
6. In the `try`:
    1. Load KnownDlls (64-bit and 32-bit) and set the user directories on `CPathResolver`.
    2. `session = new CDepends(inputFile)`, file open settings from the config.
    3. Progress callback (unless `-q`): print `[n] Analyzing: name` for each `OpeningModule`.
    4. `CDependsAnalysisContext` + `CDependsModelAnalysisRequest`.
    5. `new CDependsAnalysisService(coreClient).AnalyzeModel(request)`, running **synchronously on the main thread** (the analysis scope manages the activation context).
    6. If `rootModule.FileNotFound`, print an error and return 1.
    7. `CExporter.Export(session, outputFile, format, exportOptions)`.
7. Exit codes: 0 success; 1 for errors, missing file or failed export; 130 on cancellation (no output file is written).
8. `finally`: remove the cancel handler and `DisconnectClient()`.

11.3 Export and duplicates

`CExporter` walks `CDepends` and skips duplicates using the analysis's own marker (`OriginalInstanceId != 0`) before falling back to its first-seen set:

- `CollectAllModules` (JSON, CSV) and `CollectGraphData` (DOT) skip duplicate instances. Duplicates carry no dependents, so the canonical instance is the one to export.
- `BuildHtmlTree` and `BuildTextTree` show duplicates as leaf nodes.

This is required because the analysis chooses the canonical instance in the GUI traversal order, not in plain depth-first order.

11.4 GUI/CLI differences that remain

- No UI progress, no stats dialog, no tree.
- Console log callback instead of the rich-text log.
- No file open settings dialog; settings come from the config.
- Cancellation through Ctrl+C.

- Analysis runs on the main thread (there is no UI to keep responsive).
- Depth is controlled by `-d`.

---

12. Debug-only parity diagnostics (`#if DEBUG`)

In debug builds the form records duplicate decisions in `_duplicateObserver` while rendering, and validates them against a model-only traversal after render (`ValidateDuplicateObservations`). Mismatches are logged. Release builds contain none of this code.

---

13. Sequence summary (GUI, executable)

```
User -> OpenFileHandler
  -> OpenInputFileAsync              : guard, create CTS
  -> ExecuteAsync                    : lock UI, resolve .lnk
  -> OpenInputFileInternalAsync      : settings dialog, snapshot config, new CDepends
  -> Task.Run(AnalyzeModel)          : [worker]
        EnterAnalysis -> BeginAnalysis (caches cleared, ActCtx)
        TraverseModelCore -> per module: cancel check, duplicate or ProcessModule
              ProcessModule -> OpenModule -> headers -> imports/exports
                            -> forwarders -> stats -> CloseModule -> warnings
  <- back on UI thread
  -> copy imports table
  -> RenderAnalyzedModel -> PopulateModuleTree -> RenderAnalyzedModule
  -> fill list view, post-open updates
  -> ExecuteAsync                    : result message, unlock UI, focus tree
```

---

14. Notes and known limitations

- The render step reads `_configuration.ModuleNodeDepthMax` (live value), while the analysis uses the snapshot. Changing the setting during an analysis cannot happen (UI is locked), so they agree.
- `InstanceId` is a hash code; collision risk.
- A session load cannot be interrupted until deserialization finishes.
- `Application.DoEvents()` during the session render allows re-entrancy; the UI lock and `_openInputFileInProgress` guard against repeated opens.
- Never dispose `CCoreClient` while an analysis is active.